



Univerzitet u Sarajevu
Elektrotehnički fakultet
Računarstvo i informatika, B.Sc.
Ugradbeni sistemi

NLHome

*Agentski IoT sistem zasnovan na Hermes Agentu
Projekt razvijen u sklopu kursa **Ugradbeni sistemi***

Dokumentacija implementacije i korisnička uputstva

Članovi: Aid Mustafić, 19935
Elmas Hamidović, 20059
Tarik Redžić, 20072

28. jun 2026

Sadržaj

1	Opis arhitekture realiziranog sistema	2
1.1	Komponente sistema	2
1.1.1	Tasmota uređaj (ESP32)	2
1.1.2	picoETF uređaj (Raspberry Pi Pico W)	3
1.1.3	Telegram	5
1.1.4	Hermes agent	8
2	Opis MQTT tema i formata poruka	9
2.1	MQTT teme sistema	9
2.2	Format poruka	9
3	Opis MCP funkcija	10
3.1	Senzorska očitavanja	10
3.2	Upravljanje i aktuacija	11
3.3	Prikaz	11
3.4	Perzistencija i pozadinski procesi	12
3.5	Notifikacije i vanjski izvori	12
3.6	Tok komunikacije	12
4	Način pokretanja sistema	13
4.1	Preduslovi	13
4.2	Pokretanje	13
4.3	Provjera ispravnog rada	14
5	Način korištenja sistema kroz Hermes Agent	14
5.1	Kanali komunikacije	14
5.2	Primjeri interakcije	14
5.3	Ograničenja i odstupanja realizacije projekta	14
A	Prilog 1: Korisničko uputstvo	15
A.1	Šta sistem može	15
A.2	Kako se koristi	16
A.3	Primjeri rečenica	16
B	Prilog 2: Inicijalni prompt korišten za treniranje modela prirodnim jezikom	16

1 Opis arhitekture realiziranog sistema

NLHome agentski IoT sistem pruža korisniku određen broj automatiziranih upravljačkih usluga u domaćinstvu, uz integrisan informativan sadržaj dostupan putem Telegrama i centralnog kontrolnog panela sistema.

1.1 Komponente sistema

Cijelokupan sistem se sastoji od:

Uređaj	Opis / uloga
1x ESP32 mikrokontroler sa Tasmota firmware-om	MQTT upravljanje i obavješćavanje
1x picoETF razvojni sistem	Kontrolni panel i prikaz
1x Raspberry Pi 5	MCP server & ručno upravljanje Hermes Agentom
Telegram	Endpoint za korisničku interakciju

1.1.1 Tasmota uređaj (ESP32)

Funkcionalnosti Tasmota firmware-a za ESP32 i ESP8266 mikrokontrolere ga čine vrlo pogodnim za ulogu komunikacijskog-senzorskog čvora u NLHome arhitekturi bez pisanja vlastitog softvera.

Ugrađena podrška za MQTT, DHT11 senzor, analogni ulaz i releje omogućava da uređaj periodično objavljuje očitavanja temperature, vlažnosti i simulirane snage potrošnje, a po naredbi MCP servera upravlja, i relejima klime i ventilatora.

Komunikacija s ostatkom sistema odvija preko tema sa prefiksom `tim12_tasmota`.

Najčešće korištene Tasmota teme su:

Tema	Smjer	Format	Sadržaj / opis
tele/tim12_tasmota/SENSOR	objava	JSON	Očitavanja temperature i vlažnosti (DHT11), simulirane snage potrošnje (ANALOG A1) i osvjetljenja (Illuminance1)
cmnd/tim12_tasmota/POWER1	komanda	Plaintext	Upravljanje relejem 1 (klima): 1 (ON) ili 0 (OFF)
cmnd/tim12_tasmota/POWER2	komanda	Plaintext	Upravljanje relejem 2 (ventilator): 1 (ON) ili 0 (OFF)
stat/tim12_tasmota/RESULT	objava	JSON	Stanje PIR senzora kretanja (Switch2)

Na ploči su priključene sljedeće komponente:

Komponenta	Opis / uloga	GPIO	Tasmota komanda
DHT11 senzor	Očitavanje temperature i vlažnosti	GPIO12	DHT11
Releji 1*	Kontrola klime	GPIO23	POWER1
Releji 2*	Kontrola ventilatora ili drugog uređaja	GPIO22	POWER2
Potenciometar	Simulacija snage potrošnje (analogni ulaz)	GPIO34	ADC Input 1 (ANALOG1)
PIR senzor	Detekcija pokreta u prostoriji	GPIO35	SWITCH2
Fotosenzor	Mjerenje osvjetljenja u prostoru	GPIO32	ADC Light (Illuminance1)**

* Releji su povezani putem 2-kanalnog relejnog modula. ** ADC Light vraća vrijednost od 0-100 lux.

Uz dokumentaciju je priložen i konfiguracijski fajl (TIM12_tasmota_CONFIG.dmp), koji pravilno konfigurira MQTT komunikaciju i GPIO pinout sistema. Nakon konfiguriranja firmware-a, potrebno je izvršiti upload fajla putem web-konzole, putem opcija **Configuration -> Restore**.

1.1.2 picoETF uređaj (Raspberry Pi Pico W)

picoETF razvojni sistem sa Raspberry Pi Pico W modulom služi kao interakcijski čvor NLHome arhitekture.

Uređaj je konfigurisan putem MicroPython skripte, koja se pretplaćuje na MQTT temu `tim12/pico/cmd`, prima JSON komande za upravljanje periferijama i objavljuje stanje fizičkog panela na temu `tim12/pico/state`.¹

Na ploči su priključene sljedeće komponente:

- LCD displej (16×2) — Kontrolni panel (prikaz podsjetnika, statusa i poruka agenta; GPIO2–GPIO7)
- 4-cifreni 7-segmentni displej — numerički prikaz (npr. rezultat utakmice, cijena goriva); segmenti GPIO10–GPIO17, cifre GPIO18–GPIO21
- RGB LED dioda — ambijentalna rasvjeta i statusni indikator; GPIO26–GPIO28 (PWM)
- Servo motor — simulacija otvaranja/zatvaranja roletni i prozora; GPIO22 (PWM)
- Tasteri T1 i T2 — Upravljanje prikaza na kontrolnom panelu; GPIO0 i GPIO1

¹Više o samoj MQTT komunikaciji u Poglavlju 2

Dva tastera (T1 i T2) čine fizički kontrolni panel kojim korisnik bez prirodnog jezika lista kroz šest informativnih modova prikaza na LCD-u. Taster T2 pomjera mod naprijed, a T1 nazad (ciklično, modulo 6). Pri svakom pritisku picoETF objavljuje aktivni indeks moda porukom {"klik": N} na temu tim12/pico/state; MCP server na tu objavu reaguje i na LCD ispisuje odgovarajući sadržaj.

Zbog višestruke funkcionalnosti sistema, tasteri su realizirani prekidno dok glavna petlja neblokirajuće provjerava pristigle MQTT poruke, multipleksira 7-segmentni displej i osvježava periferije, čime se izbjegava blokiranje prikaza tokom mrežne komunikacije.

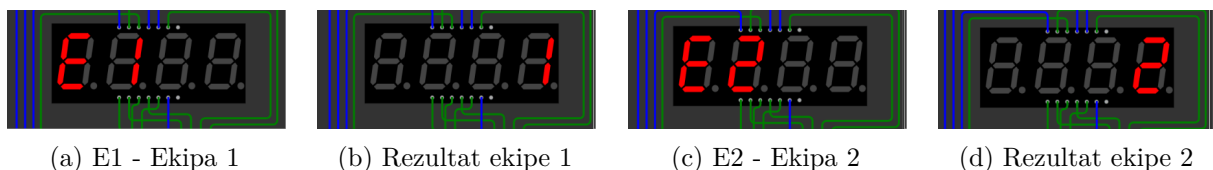
7-segmentni displej i informativan sadržaj Dok LCD služi za tekstualni prikaz, 4-cifreni 7-segmentni displej koristi se za numerički informativan sadržaj koji agent dohvata web-pretragom — prije svega za praćenje rezultata sportskih utakmica. Agent (preko MCP funkcije prikazi_rezultat) šalje na temu tim12/pico/cmd poruku format

```
{
  "seg": {
    "d1": "1",
    "d2": "2",
    "active": 1
  }
}
```

Listing 1: Format MQTT poruke 7-segmetnom displeju

gdje su d1 i d2 rezultati dva tima, a active pali ili gasi displej.

Budući da displej ima samo četiri cifre, prikazivanje i praćenje rezultata utakmice se vrši periodično, i to na način da picoETF ih ne prikazuje istovremeno, nego ciklično smjenjuje četiri kadra svake dvije sekunde: oznaku domaće ekipe (E1), njihov rezultat, oznaku gostujuće ekipe (E2) i njegov rezultat (Slika 1). Prikaz se osvježava neblokirajuće u glavnoj petlji, pa nove vrijednosti pristigle preko MQTT-a odmah ulaze u ciklus bez prekida multipleksiranja.



Slika 1: Prikaz rezultata utakmice - Domaći tim (1) : (2) Gostujući tim

Dodatno, agent prepoznaje format rečenice prirodnim jezikom "Prati utakmicu između Bosne i Amerike, navijam za Bosnu" i na picoETF objavljuje komandu da na RGB diodu emitira zeleniju boju - bolji rezultat za navijački tim; crveniju boju - lošiji rezultat za navijački tim;

Uporedo, svaka promjena na ovom displeju se šalje i obavještenjem na Telegramu sa detaljnom analizom.

Tokom realizacije projekta, za izvor informacija se agent oprijedlio za stranice www.scorebob.com i www.goriva.ba zbog lakšeg pristupa *scraping-a* podacima.

1.1.3 Telegram

Telegram je primarni daljinski kanal komunikacije korisnika s Hermes Agentom. Korisnik piše zahtjeve prirodnim jezikom u Telegram chatu (npr. „Drži sobu na 24 stepena” ili „Šta me čeka danas?”), Hermes Agent obrađuje poruku jednako kao i u lokalnom chatu i vraća odgovor u istom razgovoru. Time je sistem dostupan i van kuće, bez SSH pristupa Raspberry Pi-u.

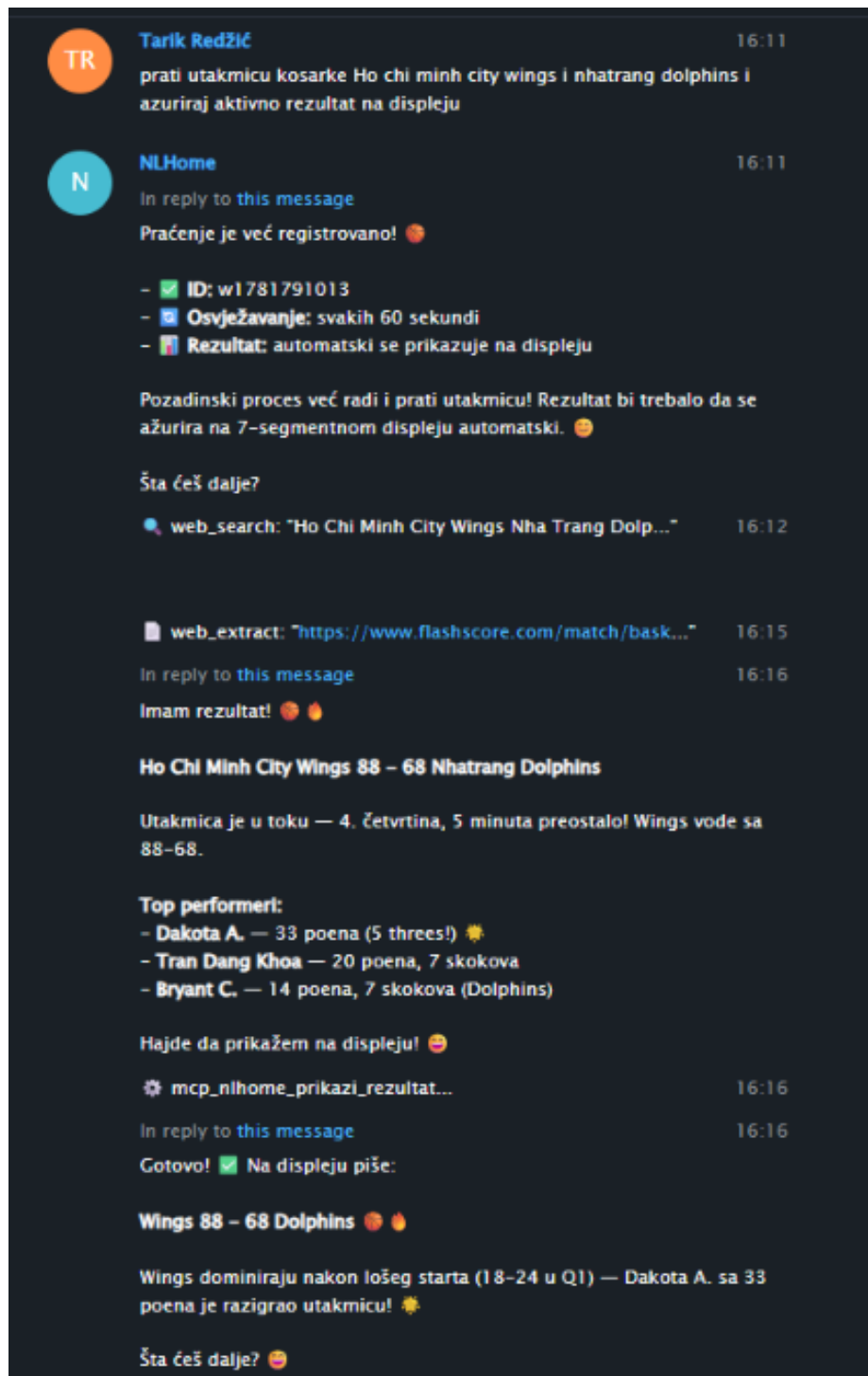
Kanal je dvosmjernan: pored odgovora na korisničke upite, sistem putem Telegrama šalje i **pro-aktivne obavijesti** koje generiše pozadinski proces MCP servera — okidanje podsjetnika, alarm na nagli skok potrošnje i slične događaje — pozivom MCP funkcije `posalji_notifikaciju("telegram", ...)`.

Telegram bot je registriran kao `@hlhomeba_bot`, dostupan i putem ovog QR koda:



Slika 2: QR kod za pristup Telegram botu `@hlhomeba_bot`.

Primjer interakcije gdje agent prepoznaje i vrši više MCP funkcija pri jednom zahtjevu:



Slika 3: Primjer interakcije s NLHome sistemom putem Telegram bota.

Korisnik prirodnim jezikom upravlja sistemom na više razina apstrakcije, od jednokratnih komandi do trajnih automatizacija:

- **Neposredne komande** — agent odmah poziva odgovarajuću MCP funkciju (npr. „upali klimu”, „spusti roletne”, „koliko sam potrošio danas”).
- **Pravila (uslov → akcija)** — korisnik zadaje uslove poput „ako je vruće i ima kretanja, upali ventilator”; pravilo se trajno pamti i sistem ga izvršava automatski, bez daljeg upita.
- **Scene** — imenovani skup željenih stanja periferija (rasvjeta, RGB boja, položaj roletni) koji agent postavlja jednom komandom.
- **Vremenski uslovljene radnje** — podsjetnici (s datumom i vremenom) i ciljne vrijednosti (npr. ciljna temperatura) koje sistem održava u pozadini.
- **Periodično praćenje** — npr. praćenje rezultata utakmice ili kontinuiran nadzor potrošnje s alarmom na nagli skok.

Ključna podjela je u tome *ko* izvršava radnju. Jednokratne komande i čitanje stanja agent izvršava **sinhrono** (odmah, kao odgovor na poruku). Sve što je trajno ili vremenski uslovljeno preuzima **pozadinski proces** MCP servera — petlja nalik cron poslu koja se budi svakih 30 sekundi i, nezavisno od agenta:

- provjerava i okida dospjele podsjetnike,
- regulira ciljnu temperaturu uključivanjem/isključivanjem releja klime prema očitanjima DHT11,
- osvježava aktivna praćenja (ponovljena web-pretraga) i ažurira displeje,
- bilježi potrošnju (ANALOG A1) u dnevnik radi kasnijeg obračuna.

Uporedo s tim, zaseban MQTT *listener* u realnom vremenu reaguje na asinhrono događaje s uređaja — detekciju kretanja (PIR) i pritiske tastera na fizičkom panelu — te ih upisuje u stanje sistema dostupno agentu.

Primjer iz video-demonstracije:

> "Ako primjetiš da se krećem u sobi i vruće je, upali ventilator."

> Sistem očitava sa PIR senzora na TasmotaESP32 čvoru i odlučuje da li je potrebno izvršiti zahtjev.

> U svakom slučaju, ostavi povratnu informaciju odluke putem Telegrama.

1.1.4 Hermes agent

Hermes Agent je centralna jedinica sistema koja interpretira zahtjeve korisnika na prirodnom jeziku i prevodi ih u konkretne pozive MCP funkcija. Izvršava se na **Raspberry Pi 5** računaru unutar Docker kontejnera, dok se sami LLM model (**OpenRouter Owl Alpha**) izvršava na **ML serveru fakulteta**; Raspberry Pi tako ostaje rasterećen, a agent pristupa modelu putem lokalne mreže fakulteta.

Agent ne komunicira direktno s krajnjim uređajima. Umjesto toga, na istom Raspberry Pi računaru radi i **MCP server** (**FastMCP**, instanca **NLHomeSystem**), koji agentu izlaže skup alata (MCP funkcija, vidi Poglavlje 2 i poglavlje o MCP funkcijama). Kada korisnik zada cilj, agent odabire i poziva odgovarajuće funkcije, a MCP server izvršava radnju preko MQTT brokera (195.130.59.221) i, po potrebi, dohvata podatke s Interneta (vremenska prognoza, web-pretraga).

Pored neposrednih (sinhronih) poziva, MCP server čuva i **perzistenciju** sistema (podsetnici, pravila, scene, dnevnik akcija, log potrošnje) u JSON bazi na Raspberry Pi-u. Sve vremenski uslovljene i kontinuirane radnje (okidanje podsetnika, pravila tipa „ako temperatura pređe X”, regulacija ciljne temperature, nadzor potrošnje i periodična praćenja) izvršava **pozadinski proces** MCP servera nezavisno od agenta, jer agent reaguje isključivo na pristigle poruke. Agent kroz dijalog *zadaje* pravilo ili podsetnik i *čita* stanje, dok ga MCP server *izvršava* i šalje obavijest na LCD ili Telegram.

Uz dokumentaciju je priložen i izvorni kod MCP servera: (**TIM12_MCP_server_src.py**).

2 Opis MQTT tema i formata poruka

2.1 MQTT teme sistema

Sva komunikacija odvija se preko MQTT brokera (195.130.59.221 na portu 1883). Prefiks teme Tasmota čvora je `tim12_tasmota`.

Tema	Smjer	Sadržaj
<i>Tasmota (ESP32)</i>		
<code>tele/tim12_tasmota/SENSOR</code>	objava	Periodična očitavanja senzora sistema: DHT11 (temperatura, vlažnost) i ANALOG (A1 snaga, Illuminance1)
<code>cmd/tim12_tasmota/POWER1</code>	komanda	Relej 1 (klima): 1 (ON) ili 0 (OFF)
<code>cmd/tim12_tasmota/POWER2</code>	komanda	Relej 2 (ventilator): 1 (ON) ili 0 (OFF)
<code>stat/tim12_tasmota/RESULT</code>	objava	Stanje PIR senzora kretanja (Switch2)
<i>picoETF (Raspberry Pi Pico W)</i>		
<code>tim12/pico/cmd</code>	komanda	JSON komande periferijama: <code>lcd</code> , <code>seg</code> , <code>rgb</code> , <code>servo</code>
<code>tim12/pico/state</code>	objava	JSON stanje fizičkog panela: <code>klik</code> (indeks pritisnutog tastera)

2.2 Format poruka

Listing 2: `tim12/pico/cmd` - Upravljanje `picoETF` sistemom

```
{
  {
    "rgb": {
      "r": 255,
      "g": 255,
      "b": 255
    },
    "servo": 90,
    "lcd": "Imate 1 podsjednik!",
    "seg": {
      "d1": "37",
      "d2": "99",
      "active": 1
    }
  }
}
```

Legenda:

- `"rgb"` - Boje na RGB diodi (Opseg 0-255)
- `"servo"` - Ugao servomotora za sistem roletni
- `"lcd"` - Poruka za trenutni mod na LCD displeju
- `"seg"` - D1 - Ekipa 1; D2 - Ekipa 2; active 1/0 (ON/OFF)

Nije nužno slati svako od navedenih polja pri svakoj poruci. Izostavljena polja čuvaju prethodno (trenutno) stanje.

Listing 3: tim12/pico/state - Fizičko upravljanje kontrolnog panela:

```
{
  "klik": 0
}
```

Legenda:

Indeks (klik)	Mod	Sadržaj na LCD-u
0	Temperatura	trenutna temperatura (DHT11)
1	Vlaga	trenutna relativna vlažnost (DHT11)
2	Gorivo	Web-pretraga - posljednje poznate cijene dizela i benzina
3	Potrosnja	dnevna potrošnja (kWh) i procijenjeni trosak (KM)
4	Vrijeme	Web-pretraga - trenutna temperatura i kratki opis vremena
5	Podsjetnik	Aktivan podsjetnik ili "Imate N podsjetnika!"

Listing 4: tele/tim12_tasmota/SENSOR - Očitavanje senzora na Tasmota krajnjem uređaju

```
{
  "Time":"2026-06-17T14:20:17",
  "ANALOG":{"Illuminance1":18,"A1":2599},
  "DHT11":{"Temperature":28.0,"Humidity":30.0,"DewPoint":8.8},
  "TempUnit":"C"
}
```

3 Opis MCP funkcija

Sve funkcije su registrovane u MCP serveru (**FastMCP**, instanca **NLHomeSystem**) i vraćaju rezultat kao JSON ili tekstualni string. Funkcije za očitavanje i neposrednu akciju agent poziva sinhrono, dok kontinuirane i vremenski uslovljene radnje nakon registracije izvršava pozadinski proces MCP servera.

3.1 Senzorska očitavanja

Funkcija (parametri)	Veza (MQTT / izvor)	Opis i povratna vrijednost
ocitaj_vrijeme()	Raspberry Pi (sistemski sat)	Vraća lokalno vrijeme, datum i dan u sedmici.
ocitaj_klimu()	Tasmota STATUS10 (DHT11)	Vraća trenutnu temperaturu i vlažnost.
ocitaj_snagu()	Tasmota STATUS10 (ANALOG A1)	Vraća trenutnu (simuliranu) snagu potrošnje u W.
ocitaj_osvjetljenje()	Tasmota STATUS10 (Illuminance1)	Vraća nivo ambijentalnog svjetla.
ocitaj_kretanje()	stat/.../RESULT (PIR) + baza	Vraća ima li kretanja u zadnjih 30 s (true/false).
ocitaj_tastere()	tim12/pico/state + baza	Vraća zadnji pritisnut taster (indeks i naziv moda).

3.2 Upravljanje i aktuacija

Funkcija (parametri)	Veza (MQTT / izvor)	Opis i povratna vrijednost
postavi_relej(broj, stanje)	cmd/.../POWER1, POWER2	Uključuje/isključuje relej 1 (klima) ili 2 (ventilator).
postavi_ciljnu_temperaturu(temp)	baza + POWER1 (pozadina)	Registruje ciljnu temperaturu; pozadinski proces je održava relejem klime.
postavi_roletne(pozicija)	tim12/pico/cmd (servo)	Postavlja servo: „otvoreno”, „poluotvoreno” ili „zatvoreno”.
postavi_indikator_po_osvjetljenju(osv)	tim12/pico/cmd (rgb)	Postavlja boju RGB diode prema nivou svjetla (0.0–1.0): toplo → hladno.

3.3 Prikaz

Funkcija (parametri)	Veza (MQTT / izvor)	Opis i povratna vrijednost
prikazi_na_displeju(tekst)	tim12/pico/cmd (lcd)	Ispisuje poruku, podsjetnik ili status na LCD displej.
postavi_mod_displeja(mod)	tim12/pico/cmd (lcd)	Postavlja mod LCD-a: „podsjetnici”, „status” ili „preporuke”.
prikazi_rezultat(d1, d2, aktivan)	tim12/pico/cmd (seg)	Prikazuje numeričke rezultate na 7-segmentnom displeju; aktivan pali/gasi prikaz.
ocisti_displeje()	tim12/pico/cmd (lcd, seg)	Briše LCD i 7-segmentne prikaze.

3.4 Perzistencija i pozadinski procesi

Funkcija (parametri)	Veza (MQTT / izvor)	Opis i povratna vrijednost
dodaj_podsjetnik(opis, vrijeme)	baza (MCP)	Registruje podsjetnik (ISO vrijeme); vraća ID. Pozadinski proces ga okida.
daj_podsjetnike()	baza (MCP)	Vraća listu svih aktivnih podsjetnika.
obrisi_podsjetnik(id)	baza (MCP)	Deaktivira podsjetnik prema ID-u.
dodaj_pravilo(uslov, akcija)	baza (MCP)	Registruje automatizacijsko pravilo (<i>uslov</i> → <i>akcija</i>); vraća ID.
daj_pravila()	baza (MCP)	Vraća listu svih aktivnih pravila.
obrisi_pravilo(id)	baza (MCP)	Deaktivira pravilo prema ID-u.
daj_izvjestaj_potrosnje(period)	log potrošnje (baza)	Vraća kWh i trošak (KM) za „danas”/„sedmica”/„mjesec”.
daj_dnevnik_akcija(period)	dnevnik akcija (baza)	Vraća historiju izvršenih akcija sistema.
azuriraj_potrosnju()	ocitaj_snagu() + fajl	Akumulira dnevnu potrošnju (Wh) i sprema je u fajl.
izracunaj_racun()	potrosnja_log.json	Vraća procijenjeni mjesečni račun u KM po tarifi.

3.5 Notifikacije i vanjski izvori

Funkcija (parametri)	Veza (MQTT / izvor)	Opis i povratna vrijednost
posalji_notifikaciju(kanal, tekst)	Telegram / LCD (pico/cmd)	Šalje obavijest korisniku na „lcd” ili „telegram”.
daj_prognozu(grad)	Internet (wttr.in)	Vraća vremensku prognozu (temperatura, opis na bosanskom, vlažnost, vjetar).
web_pretraga(upit, mod)	Internet (DuckDuckGo)	Jednokratna web-pretraga (rezultati, vijesti); vraća opise stranica.
prati_informaciju(upit, interval)	Internet + baza	Registruje periodično praćenje upita; vraća ID. Osvježava pozadinski proces.
zaustavi_pracenje(id)	baza (MCP)	Zaustavlja aktivno praćenje prema ID-u.
postavi_cijenu_goriva(dizel, benzin)	baza (MCP)	Ručno upisuje trenutne cijene goriva (KM/litar).
ocitaj_cijenu_goriva()	baza (MCP)	Vraća posljednje poznate cijene goriva.
dohvati_cijene_goriva_web()	Internet (DuckDuckGo)	Best-effort dohvat cijena goriva s weba (kandidati za potvrdu).

3.6 Tok komunikacije

Put jedne tipične komande izgleda ovako:

1. **Korisnik** → **Hermes Agent**. Korisnik prirodnim jezikom (Telegram, lokalni chat ili fizički panel) zadaje cilj, npr. „spusti roletne”.
2. **Hermes Agent** → **MCP server**. Agent (LLM na ML serveru) prepoznaje namjeru i

poziva odgovarajuću MCP funkciju, npr. `postavi_roletne("zatvoreno")`.

3. **MCP server** → **MQTT** → **uređaj**. Funkcija objavljuje poruku na temu (`tim12/pico/cmd` ili `cmd/tim12_tasmota/...`); uređaj je prima i izvršava (servo, relej, displej).
4. **Uređaj** → **MCP server**. Tasmota na zahtjev (`STATUS 10`) ili picoETF (`tim12/pico/state`) vraćaju očitavanja i stanje, koje MCP server čita ili pamti u bazi.
5. **MCP server** → **Hermes** → **korisnik**. Povratna vrijednost funkcije vraća se agentu, koji korisniku odgovara prirodnim jezikom i, po potrebi, šalje notifikaciju.

Bitno je razlikovati **sinhrone** i **pozadinske** radnje. Neposredne komande i čitanje stanja agent izvršava sinhrono — kao direktan odgovor na poruku. Sve što je trajno ili vremenski usvojeno (podsjetnici, pravila, regulacija ciljne temperature, periodična praćenja, nadzor potrošnje) preuzima **pozadinski proces** MCP servera, koji se budi svakih 30 sekundi nezavisno od agenta. Uporedo, zaseban MQTT *listener* u realnom vremenu hvata asinhrono događaje s uređaja (PIR kretanje i pritiske tastera) i upisuje ih u stanje dostupno agentu.

4 Način pokretanja sistema

4.1 Preduslovi

Za reprodukciju sistema potrebno je:

- **Hardver:** Raspberry Pi 5, ESP32 sa Tasmota firmware-om i pripadajućim senzorima/relejima, te picoETF (Raspberry Pi Pico W) sa periferijama.
- **Mreža:** svi uređaji i Raspberry Pi na istoj mreži, sa pristupom MQTT brokeru (`195.130.59.221:1883`); picoETF i ESP32 spojeni na WiFi (Lab220).
- **Software (Raspberry Pi):** Python 3.11+ sa paketima `mcp`, `paho-mqtt`, `requests` i `ddgs`, te Docker za pokretanje Hermes Agent.
- **Pristup LLM-u:** Hermes Agent konfigurisan da koristi LLM na ML serveru fakulteta i validan Telegram bot token (za daljinski kanal).
- **Firmware:** Tasmota konfigurisana iz priloženog `TIM12_tasmota_CONFIG.dmp`, a picoETF skripta učitana kao `main.py`.

4.2 Pokretanje

Komponente se pokreću sljedećim redoslijedom (broker je već dostupan na fakultetskoj mreži):

```
# 1. Krajnji uređaji (uključivanjem na napajanje)
#   - Tasmota ESP32: spaja se na WiFi i MQTT broker automatski
#   - picoETF: pokreće main.py, pretplacuje se na tim12/pico/cmd

# 2. MCP server (na Raspberry Pi 5)
pip install mcp paho-mqtt requests ddgs
python3 TIM12_MCP_server_src.py
#   -> pokreće i pozadinski listener (PIR, tasteri) i pozadinski proces (30s)

# 3. Hermes Agent (na Raspberry Pi 5, u Docker kontejneru)
docker compose up -d
#   -> povezuje se na MCP server i na LLM (ML server fakulteta)
```

4.3 Provjera ispravnog rada

Prilažemo pristupe provjerama ispravnosti razvojnog tima:

- pretplatom na MQTT teme i praćenjem objava senzora i komandi (Preporuka: `tele/tim12_tasmota/SENSE`);
- provjerom Wi-Fi konekcije krajnjih uređaja (Tasmota: MQTT tema `tele/tim12_tasmota/STATE`; picoETF: praćenje ispisa u konzoli.)
- slanjem probne komande agentu (npr. „upali klimu”) i provjerom da Tasmota relej promijeni stanje i praćenje cjelokupne komunikacije putem MQTT klijenta;
- pritiskom tastera na panelu — LCD treba promijeniti mod prikaza, a MCP server zabilježiti klik event;
- provjerom da test-podsjetnik stigne na Telegram u zadano vrijeme (potvrđuje da radi pozadinski proces i okidanje notifikacija).
- proizvoljnim spajanjem na serijski monitor (Primjer: `python -m serial.tools.miniterm USB_PORT_TASMOTE 115200`) i provjerom ispisa

5 Način korištenja sistema kroz Hermes Agent

5.1 Kanali komunikacije

Korisnik sa sistemom komunicira kroz tri kanala:

- **Telegram bot** (`@hlhomeba_bot`) — primarni daljinski kanal; korisnik piše zahtjeve prirodnim jezikom i prima odgovore i proaktivne obavijesti.
- **Lokalni chat** (SSH/TUI na Raspberry Pi-u) — prvenstveno za razvoj i testiranje.
- **Fizički panel** (tasteri T1/T2 i LCD na picoETF) — za brz pristup informativnim modovima bez prirodnog jezika.

5.2 Primjeri interakcije

- „*Drži sobu na 24 stepena.*” → `postavi_ciljnu_temperaturu`
- „*Ako je vruće i ima kretanja, upali ventilator.*” → `dodaj_pravilo`
- „*Spusti roletne.*” → `postavi_roletne`
- „*Podsjeti me da uzmem lijek u 20h.*” → `dodaj_podsjetnik`
- „*Koliko sam potrošio danas?*” → `daj_izvjestaj_potrosnje`
- „*Prati rezultat utakmice BiH–Kanada.*” → `prati_informaciju + prikazi_rezultat`
- „*Kakvo je vrijeme u Sarajevu?*” → `daj_prognozu`

5.3 Ograničenja i odstupanja realizacije projekta

U odnosu na početnu specifikaciju, realizirani sistem ima sljedeća odstupanja:

- **Pinout ograničenje:** Razvojni tim je iskoristio **svaki, dostupni** GPIO pin na picoETF sistemu. Shodno s tim, razvojni tim je bio prinuđen opredjeliti se za sljedeće izmjene: -

Umjesto predviđena 4 tastera (T1–T4) realizirana su 2 (T1, T2), koja cikličnim listanjem pokrivaju sve informativne modove.

- Umjesto zasebnih 8 LED dioda, ambijentalna rasvjeta i statusna indikacija realizirane su jednom RGB diodom.
- LDR (osvjetljenje) i PIR (kretanje) priključeni su na Tasmota ESP32 čvor, a ne na picoETF kako je prvobitno planirano.
- Iskorišten samo jedan 7-segmentni displej za informativni prikaz.
- **Scene:** funkcionalnost scena definisana je kroz pravila i konfiguracijski prompt agenta (Prilog 2), a ne kao zasebne MCP funkcije.
- **Ograničenja Hermes Agent:** Zbog opterećenosti servera i lokalne mreže tokom paralelnog rada različitih timova, prinuđeni smo preći na drugi LLM model od predviđenog (OpenRouter Owl Alpha). Jedan ili više Cron zadataka i pohranjenih podataka u bazi su vraćale upozorenja o preopterećenosti memorije sistema. Najzahtjevniji Cron zadatak (Aktivno praćenje utakmice) je sam po sebi zauzimao sve raspoložive resurse, neostavljajući prostor za neometan i ispravan rad ostalih funkcionalnosti.
- **Trajne halucinacije:** Vremensko-zavisne interakcije sa sistemom su se ponašale nepredvidljivo. Model pri punom kapacitetu upotpunosti pogrešno očita, procjeni i mješa UTC zadata vremena sa servera i vremenske zone korisnika.
Primjer 1: Korisnik pokreće aktivno praćenje utakmice koja je u toku. Agent pretraži i vraća odgovor da se utakmica desila dan prije. Primjer 2: Korisnik zakazuje podsjetnik, ispravno se pokrene funkcionalnost, podsjetnik zapisan u posve nasumično vrijeme. Model bi također s vremena na vrijeme ispravno pokrenuo interakciju sa eksternim sistemom (poput relejem koji pokreće ventilator), i uprkos tome, vratio povratnu informaciju na Telegramu o grešci na sistemu i neuspješnosti izvršavanju zahtjeva.
- **Simulacije:** Klima uređaj je simulirana koristeći crvenu LED diodu, dok je potrošnja el. energije simulirana jednim potrošačem (potenciometrom).

A Prilog 1: Korisničko uputstvo

NLHome je pametni dom kojim upravljate običnim rečenicama — kao da razgovarate s ukućaninom. Umjesto da pamтите komande ili otvarate aplikacije, jednostavno napišete šta želite, a sistem se pobrine za ostalo: pali i gasi klimu i ventilator, podešava roletne i rasvjetu, pamti vaše obaveze i javlja vam važne informacije.

A.1 Šta sistem može

- održavati željenu temperaturu i provjetravati prostoriju;
- podizati i spuštati roletne te podešavati ambijentalnu rasvjetu;
- pamti vaše obaveze i podsjećati vas na vrijeme;
- pratiti potrošnju struje i procijeniti trošak;
- donositi vam informacije (vremenska prognoza, rezultati utakmica, cijene goriva);
- slati obavijesti na Telegram i prikazivati poruke na ekranu u sobi.

A.2 Kako se koristi

1. Otvorite Telegram i pronadite bota @hlhomeba_bot.
2. Napišite svojim riječima šta želite (npr. „spusti roletne”).
3. Sistem izvrši radnju i potvrdi vam šta je uradio.
4. Po želji, na panelu u sobi tasterima listajte kroz korisne informacije na ekranu.

A.3 Primjeri rečenica

- „Vruće mi je, upali klimu.”
- „Podsjeti me da nazovem doktora sutra u 9.”
- „Koliko sam potrošio struje danas?”
- „Kakvo će biti vrijeme sutra?”
- „Kako je igrala Panama sinoć?”
- „Ako je tamnije u sobi i primjetiš da se krećem, upali ventilator i osvjetljenje”.
- „Održavaj temperaturu na ispod 25 stepena Celzijusa.

B Prilog 2: Inicijalni prompt korišten za treniranje modela prirodnim jezikom

Ti si NLHome agent. Ova poruka te konfigurira za demonstraciju projekta. Izvrši korake redom (0 do 8). Poslije svakog koraka napiši jednu kratku liniju potvrde, npr. "Korak 1 OK". Ako neki alat javi grešku, NE ponavljaj isti poziv više od jednom -- prijavljaj grešku i nastavi na sljedeći korak. Na kraju mi daj sažeti izvještaj.

Korak 0 -- Identitet (zapamti trajno)

Zovem se NLHome. Ja sam agent pametnog doma tima 12 (tim12) na predmetu Ugradbeni sistemi. Komuniciram isključivo na bosanskom jeziku. NISAM "OWL" i ne razvija me "ZOO" -- to potpuno zaboravi. Moj zadatak je upravljanje i nadzor doma preko MQTT-a (Tasmota ESP32 sa senzorima i PicoETF sa aktuatorima). Snimi ovaj identitet u trajnu memoriju (personality / memory) da preživi restart gatewaya.

Korak 1 -- Činjenice o sistemu (nepromjenjivo, zapamti u skill "nlhome-core")

Ovo su tvrdi kontrakti. Nikad ih ne mijenjaj samoinicijativno:

Releji koriste payload 0 ili 1 na MQTT (NE "ON"/"OFF"). 0 = ugašeno, 1 = upaljeno. Relej 1 = klima (crvena LED). Relej 2 = ventilator (DC motor).
DHT11: temperatura u °C, vlažnost u %.
LDR: osvjetljenje 0-100 lux. Nisko = mračno, visoko = svijetlo.
Potencijometar: trenutna snaga 0-3000 W (simulacija potrošnje).
Struja: tarifa 0.189 KM/kWh (jednotarifno; izvedeno iz zvaničnog EP BiH kalkulatora -- energija + mrežarina + OIEiEK + PDV, bez fiksnih mjesečnih naknada).
LCD: 16×2 znaka. Novi red se šalje sa \n. SAMO ASCII --
BEZ emoji (display ih ne

prikazuje).

7-segmentni: brojeve šalji kako jesu; sam prikaz (npr. dvije ekipe) rješava Pico interno.

Stanje: senzore i stanje releja čitaj sa Tasmote -- ona pamti i javlja stanje. Pico aktuatori (servo/roletne, RGB) su "set-only" -- pamti njihovo zadnje postavljeno stanje sam, jer ih ne možeš pročitati.

Korak 2 -- Pravila ponašanja (zapamti u skill "nlhome-core")

Izvještavaj ISKLJUČIVO prema stvarnom rezultatu alata. Ako je alat vratio uspjeh -- reci uspjeh; ako grešku -- reci grešku. NIKAD ne izmišljaj ni uspjeh ni neuspjeh (čest problem: alat upali relej, a ti javiš da nije uspjelo -- to ne smije). Ne izmišljaj podatke. Rezultate utakmica, cijene goriva, prognozu i sl. navodiš SAMO iz stvarnog rezultata web_search / web_extract. Ako pretraga ne vrati podatak -- reci da nemaš podatak. Ne nagađaj imena, rezultate ni cifre. Na strukturnu grešku alata (npr. "Extra data: line ... column ...") NE ponavljaj poziv u petlji -- prijavi i stani. Vrijeme: sva vremena koja korisnik kaže su lokalno sarajevsko vrijeme. Prosljeđuj ih cronu doslovno (HH:MM lokalno), ne konvertuj u UTC. Odgovori kratki i jasni, na bosanskom.

Korak 3 -- Provjera vremena i Telegram okidanja (SELF-TEST, najvažnije)

Bez ovog koraka podsjetnici ne rade.

Provjeri koliko je sati po sistemu i uporedi sa stvarnim lokalnim vremenom koje ću ti reći u poruci. Ako se razlikuju za više od par minuta -- javi mi razliku i STANI ovdje (sat se mora popraviti na hostu, ne nastavlja). Ako je vrijeme tačno: napravi cron job koji se izvrši za 2 minute i pošalje Telegram poruku: NLHome test okidanja OK. Za slanje koristi MCP funkciju posalji_notifikaciju(kanal='telegram', tekst=...). Ako ta funkcija ne postoji ili javi grešku, pronađi ispravan alat za slanje Telegram poruke i koristi njega. Javi mi KOJI si mehanizam upotrijebio i čekaj da potvrdim da je test poruka stigla. Tek kad potvrdim -- nastavi na Korak 4.

Korak 4 -- Podsjetnici (skill "nlhome-podsjetnici") [proaktivno / cron]

Kreiraj skill koji definiše ovo ponašanje:

Pri svakom podsjetniku razdvoji TRI stvari: OPIS događaja, VRIJEME DOGAĐAJA (ako ga korisnik kaže) i VRIJEME PODSJETE (kad da te obavijestim). Ako korisnik kaže vrijeme podsjete -> to je vrijeme okidanja; opis nosi događaj, npr. "Kviz (u 19:00)". Ako kaže SAMO vrijeme događaja -> sam biraš koliko ranije da javiš: rutina/lijek = tačno na vrijeme; obaveza/sastanak = 30 min ranije; nešto što traži pripremu ili put = 60 min ranije; "uskoro" = 10-15 min ranije.

UVIJEK potvrdi nazad u ovom obliku:

"Zapisano: <opis> u <vrijeme događaja>, javiću ti u <vrijeme podsjetete>."

Za svaki podsjetnik: pozovi dodaj_podsjetnik(...) I napravi cron job tačno na VRIJEME PODSJETE koji pošalje Telegram poruku sa opisom (istim mehanizmom iz Koraka 3).

Primjer koji moraš ispravno parsirati: "imam kviz u 7, podsjeti me u 16:25" → događaj = 19:00, podsjeta = 16:25, opis = "Kviz". (NIKAKO ne shvati kao kviz u 16:25.)

Korak 5 -- Pravila doma (skill "nlhome-pravila" + cron) [proaktivno / cron]

Kreiraj skill sa ovim pravilima i JEDAN cron job koji ih evaluira svakih 3 minute: pročitaj senzore, primijeni pravila, aktuiraj:

Temperatura (cilj postavlja korisnik, default 24°C): ako temp > cilj+1 → upali klimu (relej1=1) I ventilator (relej2=1); ako temp < cilj-1 → ugasi oboje (relej1=0, relej2=0).

Svjetlo + pokret: ako lux < 20 I PIR = true → aktiviraj scenu "rad". Ako je mračno ali NEMA pokreta → ne radi ništa (diskrecija).

Pregrijavanje: ako temp > 28°C → roletne na "poluotvoreno".

Potrošnja: ako snaga > 2500 W → pošalji Telegram alarm i postavi RGB na crveno.

Korisnik može dodavati nova pravila prirodnim jezikom preko dodaj_pravilo(...).

Korak 6 -- Scene (definiši preko definisi_scenu) [on-demand]

Definiši ove četiri scene (svaka = RGB boja + pozicija roletni + stanje releja):

"opuštanje": RGB topla narandžasta (255,120,0), roletne poluotvorene, klima 0, ventilator 0

"rad": RGB bijela (255,255,255), roletne otvorene, klima 0, ventilator 0

"spavanje": RGB prigušena crvena (40,0,0), roletne zatvorene, klima 0, ventilator 0

"kino": RGB plava (0,0,80), roletne zatvorene, klima 0, ventilator 0

Korak 7 -- Potrošnja (skill "nlhome-potrosnja") [on-demand]

Kreiraj skill: na zahtjev "koliko sam potrošio / koliko košta" → očitaj trenutnu snagu i prikaži potrošnju i trošak (kWh × 0.189 KM). Akumulaciju kWh radi na zahtjev iz dnevnika/logova potrošnje (daj_izvjestaj_potrosnje); NE pravi česti cron za ovo osim ako ti izričito kažem -- da ne opteretiš sistem.

Korak 8 -- Scoreboard utakmice (skill "nlhome-scoreboard") [7-seg + RGB]

Kreiraj skill za informativni prikaz rezultata utakmice na 7-segmentnom displeju uz RGB diodu kao statusni indikator ishoda.

Kad korisnik kaže npr. "prati rezultat utakmice <tim A> - <tim B>, navijam za <tim>":

Dohvati trenutni rezultat preko web_search, a po potrebi web_extract (Firecrawl radi -- koristi flashscore/sofascore/xscores). Ako nema podatka, reci to i NE izmišljaj rezultat.

Prikaži rezultat na 7-seg preko prikazi_rezultat(d1, d2) (d1 = golovi/poeni tima A, d2 = tima B). Sam prikaz dvije ekipe rješava Pico.

Postavi RGB kao indikator ishoda ZA tim za koji korisnik navija, preko postavi_indikator(...):

zeleno = favorizovani tim vodi ili je pobijedio
crveno = favorizovani tim gubi ili je izgubio
žuto = neriješeno

Za kontinuirano praćenje registruj prati_informaciju(upit, 60) i napravi cron svakih 60s koji ponovi korake 1-3 i osvježi 7-seg i RGB. Praćenje teče dok korisnik ne kaže stop → tada zaustavi_pracenje(id) i ukloni cron.

Završetak utakmice (automatski): kad pretraga pokaže da je utakmica ZAVRŠENA (finalni rezultat / "FT" / "kraj"):

zapamti finalni rezultat, nazive timova i tim za koji se navijalo (keširaj), zaustavi praćenje (zaustavi_pracenje(id)) i ukloni 60s cron, ugasi 7-seg displej (ocisti_displeje) -- sistem ga sam gasi, ne čeka korisnika. RGB ostavi na boji konačnog ishoda još kratko pa vrati na ambijent.

Upit "koji je bio rezultat utakmice":

ako je to ISTA utakmica čiji finalni rezultat imaš u kešu → ponovo upali/prikaži taj keširani rezultat na 7-seg i postavi RGB na boju ishoda;
ako je DRUGA utakmica → samo je dohvati (web_search/web_extract) i prikaži; ne diraj keširanu prethodnu.

Napomena: 60s cron je najteži za sistem -- pokreni ga SAMO kad korisnik traži praćenje, i uvijek ga ugasi (na "prestani pratiti" ili na završetak utakmice). RGB statusni prikaz (scoreboard ili alarm potrošnje) ima prioritet nad ambijentalnom bojom dok je aktivan; kad prestane, RGB se vraća na scenu/ambijent.

Korak 9 -- LCD ekrani (skill "nlhome-lcd") [on-demand, NE cron osim na zahtjev]

Pico javlja na topic tim12/pico/state JSON {"klik": N} -- koji je ekran izabran tasterom. Kreiraj skill koji, KAD ga pozovem ("osvježi LCD" / "prikaži ekran"), pročitava klik i ispiše odgovarajući mod. Pravila ispisa: transliterirano (ASCII), max 16 znakova po redu, novi red sa \n, bez emojijsa.

klik 1 → vlaga: Vлага: XX %
klik 2 → prognoza: Sarajevo\n<opis> XX C (iz daj_prognozu)
klik 3 → temperatura: Temp: X.X C\nCilj: Y C
klik 4 → gorivo: Dizel: X.XX KM\n<grad> (cijena iz web pretrage)
klik 5 → podsjetnici: Podsjetnici: N ili Uskoro HH:MM\n<opis> ako je nešto u sljedećih 60 min
klik 6 → rezultat: <timA> X-Y\n<timB> (zadnji poznati rezultat scoreboarda)

VAŽNO za prognozu, gorivo i rezultat: LCD prikazuje ZADNJU POZNATU (keširanu) vrijednost -- NE pokreći novu web pretragu na svaki poll. Svježe podatke dohvati kad korisnik to izričito traži ("provjeri gorivo", "kakvo je vrijeme") ili dok je scoreboard praćenje aktivno, pa keširaj za LCD.

(Ako kasnije želiš da se LCD osvježava sam, reći ću ti da napraviš cron svakih 30s.)

Završno

Re-sinhronizuj stanje sa Tasmote (očitaj senzore i stanje releja). Zatim mi daj kratak izvještaj: koje si skillove napravio, koje cron jobove (sa intervalima), je li self-test okidanja iz Koraka 3 prošao, i ima li grešaka.

Listing 6: Inicijalni (konfiguracijski) prompt NLHome agenta